VERIQTA

AI for DevOps Engineers:

The Practical Handbook

Use AI to Build, Automate, Troubleshoot and Operate Real-World Infrastructure



PRACTICAL EXAMPLES | **REAL COMMANDS** | **PRODUCTION SCENARIOS** | **BEGINNER FRIENDLY** | **ENGINEERING BEST PRACTICES**

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

1. Where AI Actually Fits in DevOps

VERIQTA

1. What is AI in DevOps?

AI helps DevOps engineers understand, build, automate, troubleshoot and operate infrastructure using natural language, analysis and code generation.

2. Why DevOps Engineers Use AI

- Understand complex systems faster
- Write and improve code and configurations
- Automate repetitive tasks
- Analyze logs and metrics
- Troubleshoot issues and find root causes
- Reduce time spent on manual research

3. Where AI Fits in the DevOps Lifecycle

PLAN	CODE	BUILD	DEPLOY	OPERATE
• Research • Architecture • Design • Documentation	• Explain code • Generate code • Review changes • Fix bugs	• Dockerfiles • CI/CD configs • Scripts • Automation	• Kubernetes YAML • Terraform files • Deployment steps • Rollback guidance	• Monitor systems • Analyze logs • Investigate alerts • Incident response

4. AI Assistant vs Automation System

AI Assistant

- You ask questions
- Understands context
- Generates suggestions
- Explains solutions
- Helps you learn
- You decide what to run

AI assistant suggests and explains. You review, verify and execute.

Automation System

- Runs predefined tasks
- Follows scripts or rules
- Works without human input
- Executes actions automatically
- Does not reason or explain
- Often used in CI/CD, IaC, monitoring

Automation system executes tasks. It does not think, explain or learn.

5. Practical Example: Investigate a Failing Deployment

1. Provide context to AI
- What you deployed
- Expected behaviour
- Error messages
- Environment details

→

2. AI analyzes and suggests
- Possible causes
- Diagnostic commands
- What to check in logs
- Next steps

→

3. You run safe read-only commands
- Check pod status
- View logs
- Inspect configuration
- Verify environment

→

4. Identify and fix the issue
- Confirm root cause
- Apply the correct fix
- Test in non-production
- Deploy or rollback

6. Junior Engineer Insight

AI can suggest actions, but the engineer remains responsible for verification.

Always review AI recommendations, understand the impact, and test changes before running them in production.

⚠ 7. Important Reminders

- Do not give AI access to production systems.
- Never share sensitive data or credentials.
- Validate every suggestion.
- Use AI to support your thinking, not replace it.
- Always test changes in a safe environment first.

2. Build Your AI-Powered DevOps Workstation

VERIQTA

1. What You Need

Set up a development environment with the tools DevOps engineers use, plus an AI coding assistant.

- **VS Code** — Code editor
- **Git** — Version control
- **Linux** — Terminal
- **AI Assistant** — Coding & troubleshooting

2. Set Up an AI Coding Assistant

A Install and Configure in VS Code

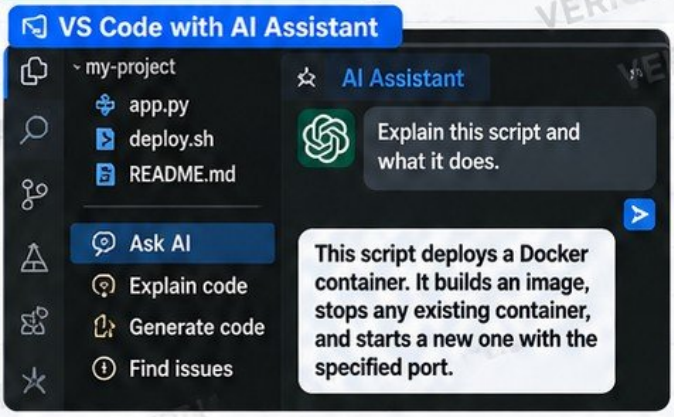
1. Open VS Code
2. Install an AI extension (e.g. GitHub Copilot, ChatGPT, or Continue)
3. Sign in and enable in your workspace
4. Test with a simple question

B Use AI in the Terminal

1. Use a terminal-integrated assistant (e.g. GitHub Copilot CLI or ChatGPT CLI)
2. Ask questions directly from your Linux terminal
3. Get command explanations and troubleshooting help
4. Keep sensitive data out of the terminal input

3. How It Works

VS Code with AI Assistant



Terminal with AI Assistant

```
user@linux:~$ ai "explain this command"
docker run -d -p 80:80 myapp:latest
```

This command:

- Runs a container in detached mode (-d)
- Maps port 80 on the host to port 80 in the container (-p 80:80)
- Uses the image myapp:latest
- Useful for running a web application

4. Understand AI Interfaces and Context

Chat Interface

- Ask natural language questions
- Get explanations and examples
- Useful for learning and debugging

Repository Context

- AI can read your code files
- Helps with code explanations, reviews and improvements
- Works best with clear context

Terminal-Integrated

- Use AI directly in your Linux terminal
- Get command explanations
- Troubleshoot errors quickly
- Save time during day-to-day work

5. Practice: Explain a Bash Script

```
1  #!/bin/bash
2  LOG_DIR=/var/log/myapp
3  if [ ! -d "$LOG_DIR" ]; then
4     mkdir -p "$LOG_DIR"
5  fi
```

Ask AI: "Explain this script"

- What does each line do?
- Why is the check needed?
- When would you use this?
- Are there any improvements?
- Test the script and verify it works.

6. Security Reminder

- Do NOT paste production credentials.
- Do NOT share private keys.
- Do NOT share sensitive logs, customer data or internal code.
- Use approved AI services only.
- Review AI suggestions before running any command.

7. Junior Engineer Insight

AI can speed up learning and help you troubleshoot, but you must always verify the output, understand the impact, and make the final decision.

VERIQTA

3. Give AI the Right Engineering Context

1. Why Context Matters

- Vague prompts give unclear or generic answers.
- AI cannot see your environment unless you provide details.
- Missing information leads to wrong recommendations.
- Good context helps AI give accurate, safe and practical solutions.

2. Key Information to Provide

Operating system
e.g. Ubuntu 22.04, CentOS 8, Amazon Linux

Runtime / version
e.g. Docker 24.0, Python 3.11, Kubernetes 1.28

Configuration
e.g. Dockerfile, YAML, environment variables

Error output
Full error message and relevant logs

Expected behavior
What should happen if things work correctly

Constraints
e.g. no internet access, specific versions, company policies, limited permissions

3. Vague Prompt vs Structured Prompt

Vague Prompt (Not Helpful)

> "Docker is broken.
> It's not working.
> How do I fix it?"

- No environment details
- No error message
- No expected behavior
- Leads to generic answers
- May suggest wrong fixes

Structured Prompt (Better)

> "I'm running Docker on Ubuntu 22.04.
> When I run 'docker run -d -p 80:80 nginx'
> I get this error: (paste error message).
> Docker version is 24.0. I expect the container to start and be accessible on port 80.
> What could be the cause and how do I fix it?"

- Provides OS and Docker version
- Includes exact error message
- States expected behavior
- Gives clear context and goal
- Helps AI give accurate, step-by-step guidance

4. Practical Example: Fix a Docker Issue

1 Step 1: Provide context

- OS: Ubuntu 22.04
- Docker version: 24.0
- Command: docker run -d -p 80:80 nginx
- Error: bind: address already in use
- Expected: container should start
- Ask for cause, diagnostic commands and safe fix.

2 Step 2: Useful AI response

AI suggests:

1. Check what is using port 80
   ```
   sudo lsof -i :80
   ```
2. Stop the conflicting service
   ```
   sudo systemctl stop nginx
   ```
3. Or use a different port
   ```
   docker run -d -p 8080:80 nginx
   ```
4. Verify it is running
   ```
   docker ps
   ```

3 Step 3: You verify

- Run the suggested commands.
- Check the actual output.
- Confirm the fix works (container is running and accessible).
- Do not run destructive commands without understanding the impact.

5. What to Ask AI For

- Explain the problem and possible causes
- Provide diagnostic commands with explanation
- Suggest multiple solutions (with pros and cons)
- Include verification steps
- Highlight any risks or things to check first

6. Important Reminders

- Never paste production credentials, private keys or sensitive logs into unapproved AI services.
- Review AI suggestions before running any command.
- Validate that the solution matches your environment.
- AI can make mistakes. Always verify.
- Use non-production environments when possible.

7. Junior Engineer Insight

The quality of AI's answer depends on the quality of your question.
Be specific, provide real context and always verify the output.

4. Troubleshoot Linux with AI and Real Evidence

VERIQTA

1. Common Linux Problems

- High CPU usage (system is slow)
- Memory pressure (applications killed)
- Failed services (not running or restarting)
- Disk exhaustion (no space left)

2. Useful Commands for Investigation

top	View real-time CPU and process usage
free -h	Check memory usage
df -h	Check disk space usage
systemctl status	Check the status of a service

3. Investigate High CPU Usage

```
$ top
top - 14:10:31 up 1 day,  2:10,  2 users,  load average: 2.5
Tasks:  204 total,   2 running, 202 sleeping,    0 stopped
%Cpu(s): 85.1 us,  5.2 sy,   0.0 ni,  9.0 id,  0.7 wa,  0.0 hi
  PID  USER      PR  NI    VIRT    RES    SHR  S  %CPU  %MEM
 2487  app       20   0  482312  120832   8620 R  78.4   4.1
 1120  root      20   0  125648   32048   8040 S  12.3   1.0
 3321  app       20   0  225432   45220   9332 S   5.6   1.5
```

How to analyze

- Look at the %CPU column.
- Identify the process using the most CPU.
- Check what the process is doing:

```
ps -p <PID> -o pid,cmd
```

- Decide if it is expected or a problem.

4. Check Memory Usage

```
$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.7G        6.1G        512M        320M        1.1G        1.2G
Swap:          2.0G        512M        1.5G
```

How to analyze

- If free memory is very low, applications may be killed.
- Check which processes are using memory:

```
ps aux --sort=-%mem | head
```

- Look for memory leaks or misbehaving apps.

5. Check Disk Space

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   46G  2.1G  96% /
tmpfs           7.7G   12M  7.7G   1% /dev/shm
/dev/sda2       100G   20G   75G  22% /var
/dev/sdb1       200G  180G  5.2G  97% /data
```

How to analyze

- Look at the Use% column.
- If a partition is 90–100%, applications can fail to write logs or files.
- Find large files:

```
sudo du -h /var | sort -h | tail
```

- Clean unnecessary files or increase storage.

6. Check Service Status

```
$ systemctl status myapp
● myapp.service - My Application
   Loaded: loaded (/etc/systemd/system/myapp.service; enable
   Active: failed (Result: exit-code) since Thu 2024-01-25
   Process: 2487 ExecStart=/opt/myapp/start.sh (code=exited,

Jan 25 14:10:31 server systemd[1]: Started My Application.
Jan 25 14:10:35 server myapp[2487]: Error: cannot connect t
Jan 25 14:10:35 server systemd[1]: myapp.service: Main proc
Jan 25 14:10:35 server systemd[1]: myapp.service: Failed wi
```

Common service states

- active (running) – service is working
- activating – starting
- failed – service failed to start
- inactive – stopped

How to investigate

- Read the error message in the output.
- Check application logs:

```
sudo journalctl -u myapp -n 50
```

- Fix the underlying issue, then restart:

```
sudo systemctl restart myapp
```

7. Practical Example: Service Keeps Restarting

Service restarts continuously	Check service status and logs	Identify root cause	Fix the issue and verify
• Use systemctl and logs	• Look for error messages	• Check config, dependencies, permissions, disk or memory issues	• Restart service and confirm it stays running

8. Junior Engineer Insight

- AI can help you analyze logs, explain errors, and suggest fixes.
- Always verify AI suggestions with real system state and logs.
- Do not apply changes blindly.
- Understand why the issue happened.
- Confirm the service stays stable after fixing it.

5. Generate Bash Scripts Without Creating Risks

VERIQTA

1. Why Use AI for Bash Scripts?

- Quickly draft scripts for common tasks
- Explain and improve existing scripts
- Add error handling and logging
- Help with best practices and security
- Save time while learning

2. Common Use Cases

Log inspection
Search, filter and analyze logs

Disk monitoring
Check disk usage and alert

Routine checks
Service status, backups, environment health

Automation
Run scheduled tasks with cron

3. Key Bash Concepts

- **Variables** – store values (e.g. DISK_THRESHOLD=80)
- **Quoting** – prevents errors with spaces and special characters
- **Exit codes** – 0 = success, non-zero = error
- **Error handling** – detect and handle failures
- **Logs** – record output for troubleshooting

4. Exit Codes (Common Examples)

Exit Code	Meaning
0	Success
1	General error
2	Incorrect usage (e.g. wrong arguments)
126	Permission denied (not executable)
127	Command not found

5. Practical Example: Disk-Usage Alert Script

```bash
#!/bin/bash
# Disk usage alert script

THRESHOLD=80
PARTITIONS="// /var /home"

for p in $PARTITIONS; do
    USAGE=$(df -h $p | awk 'NR==2 {print $5}' | tr -d '%')
    if [ "$USAGE" -ge "$THRESHOLD" ]; then
        echo "WARNING: $p is ${USAGE}% full"
        # Optional: send email or notification here
        exit 1
    else
        echo "$p usage is ${USAGE}% (OK)"
    fi
done

exit 0
```

How It Works

1. Sets a disk usage threshold (80%)
2. Checks selected partitions
3. Extracts the current usage percentage
4. If usage is above the threshold, shows a warning and exits with code 1
5. If all partitions are OK, exits with code 0

Run the Script

1. Save as disk_check.sh
2. Make it executable:
   `chmod +x disk_check.sh`
3. Run it: `./disk_check.sh`

6. Use AI to Improve or Explain the Script

- Ask AI to explain each line of the script
- Request improvements (e.g. email alerts, log file)
- Add error handling and input validation
- Test different scenarios (high usage, low usage)
- Ask for best practices and security checks

7. Common Beginner Mistakes

- Running AI-generated scripts with sudo without reviewing them
- Not understanding what the script does
- No error handling or logging
- Incorrect quoting (causes unexpected behaviour)
- Ignoring exit codes
- Hardcoding sensitive information

8. Junior Engineer Insight

AI can generate scripts quickly, but you must always read, understand and test the script before running it, especially in production environments.

9. Security Reminder

- Never run untrusted scripts with sudo.
- Do not paste production credentials, private keys or sensitive logs into an unapproved AI service.
- Review and test scripts in a safe environment first.

6. Use AI to Write and Debug Python Automation

VERIQTA

1. What You Will Learn

- Generate a Python script that checks an HTTP endpoint.
- Understand timeouts, exceptions, response codes, and structured output.
- Return a useful process exit code.
- Test successful responses, failures, and unreachable services.

2. Python Concepts Used

- **Requests:** Make HTTP requests (use requests library)
- **Timeouts:** Prevent hanging requests
- **Exceptions:** Handle network and HTTP errors
- **Response codes:** Check if service is healthy
- **Structured output:** Print clear and readable results
- **Exit codes:** Return 0 (healthy) or non-zero (unhealthy)

3. Example: Service Health Check Script

```python
#!/usr/bin/env python3
# Simple HTTP endpoint health check
import sys
import requests
import json

URL = sys.argv[1] if len(sys.argv) > 1 else "http://localhost"
TIMEOUT = 5

try:
    response = requests.get(URL, timeout=TIMEOUT)
    status = response.status_code
    result = {
      "service": URL,
      "status_code": status,
      "response_time": response.elapsed.total_seconds(),
      "healthy": 200 <= status < 400
    }
    print(json.dumps(result, indent=2))
    if 200 <= status < 400:
        sys.exit(0)  # healthy
    else:
        sys.exit(1)  # unhealthy (HTTP error)

except requests.exceptions.Timeout:
    print(json.dumps({"service": URL, "error": "timeout",
      "message": f"Request timed out after {TIMEOUT} seconds",
      "healthy": False}, indent=2))
    sys.exit(2)  # timeout
except requests.exceptions.RequestException as e:
    print(json.dumps({"service": URL, "error": "connection_error",
      "message": str(e), "healthy": False}, indent=2))
    sys.exit(3)  # connection error
```

4. How to Run the Script

1. Install requests library (if not installed)
   `pip install requests`
2. Save the script as check_service.py
3. Make it executable
   `chmod +x check_service.py`
4. Run the script with a URL
   `./check_service.py https://example.com`

5. Example Outputs

Successful Response (exit code 0)

```
$ ./check_service.py https://example.com
{
  "service": "https://example.com",
  "status_code": 200,
  "response_time": 0.321,
  "healthy": true
}
```

Exit code: 0 (healthy)

HTTP Error (exit code 1)

```
$ ./check_service.py https://example.com/invalid
{
  "service": "https://example.com/invalid",
  "status_code": 404,
  "response_time": 0.210,
  "healthy": false
}
```

Exit code: 1 (unhealthy)

Timeout (exit code 2)

```
$ ./check_service.py https://10.255.255.1
{
  "service": "https://10.255.255.1",
  "error": "timeout",
  "message": "Request timed out after 5 seconds",
  "healthy": false
}
```

Exit code: 2 (timeout)

6. Test Different Scenarios

Scenario	Example URL	Expected Result	Exit Code
Healthy service	https://example.com	healthy: true	0
HTTP error (404)	https://example.com/invalid	healthy: false	1
Timeout	https://10.255.255.1	timeout error	2
Connection error	http://notarealhost12345.com	connection error	3

7. Important Reminders

- Use a timeout to avoid hanging scripts.
- Handle exceptions to prevent crashes.
- Check and return meaningful exit codes.
- Do not expose sensitive data in logs.
- Test in different scenarios (success, HTTP error, timeout, unreachable).
- Review AI-generated code before running it.

8. Junior Engineer Insight

AI can quickly generate useful scripts, but always read, understand, test and modify the code to fit your environment. Verify the output and exit codes work as expected.

7. Review Git Changes and Pull Requests with AI

VERIQTA

1. What You Will Learn

- Use AI to explain unfamiliar code changes and identify risky modifications.
- Analyze git diff, commit history, and pull request changes.
- Review a proposed change to an application deployment script.
- Understand why AI review complements, rather than replaces, testing and human review.

2. Useful Git Commands

Command	Purpose
git status	Check working directory status
git diff	View changes in files
git diff --staged	View staged changes
git log --oneline	View commit history (short format)
git show <commit>	See details of a specific commit
git branch -a	List all branches
git remote -v	View remote repositories

3. Example: View Changes with git diff

```
$ git diff
diff --git a/deploy.sh b/deploy.sh
index 3f2a1c4..9b7d8e1 100755
--- a/deploy.sh
+++ b/deploy.sh
@@ -5,7 +5,7 @@
-   echo "Stopping application..."
+   echo "Stopping application and backing up..."
    docker stop myapp
    docker rm myapp

-   docker run -d --name myapp myapp:latest
+   docker run -d --name myapp \
+       -e ENV=production \
+       -p 8080:80 myapp:latest
```

4. Analyze Commit History

```
$ git log --oneline --graph --decorate -5
* 9b7d8e1 (HEAD -> main, origin/main) Add
  environment variable and update run command
* 3f2a1c4 Fix typo in health check
* ac4e9d2 Update deployment script
* 7d1e9b0 Improve logging
* 5a2c7f1 Initial commit
```

5. Review a Pull Request with AI

1. Provide the PR description, changed files, and diff to AI.
2. Ask AI to explain what changed and why.
3. Request a summary of potential risks.
4. Ask for things to test and verify before merging.
5. Review AI's findings and make your own decision.

6. Example AI Review of a Deployment Script Change

You ask AI:

"Review this git diff for a deployment script. Explain what changed, the potential risks, and what I should test before merging."

AI Response (Example)

Summary:

- The script now backs up the application before stopping it.
- An environment variable ENV=production is added.
- The container is explicitly mapping port 8080 to 80.

Potential Risks:

- If the backup step fails, the deployment may stop.
- ENV=production may change application behavior.
- Port mapping may conflict if 8080 is already in use.

Recommended Tests:

- Run the script in a staging environment.
- Verify the backup is created.
- Check the application starts with the new environment variable.
- Confirm the container is accessible on port 8080.

7. Identify Risky Changes

- Review changes to environment variables.
- Check for removal of security settings.
- Look for hardcoded credentials or sensitive data.
- Identify changes that affect production infrastructure.
- Verify that logging, error handling, and rollback steps exist.

8. Junior Engineer Insight

AI can help you understand code changes and spot risks, but it is not a replacement for testing, staging, and human review. Always verify AI's explanation and test the changes before merging.

9. Important Reminders

- Do notmerge changes based only on AI's recommendation.
- Always test in a non-production environment first.
- Check for security and compliance issues.
- Review the full context and have another engineer review important changes.
- Use AI as an assistant, not the final decision maker.

8. Build Better Dockerfiles with AI

VERIQTA

1. What You Will Learn

- Ask AI to review Dockerfile layers, base images, dependencies, and build context.
- Identify unnecessary packages, root execution, and oversized images.
- Improve a Python application's Dockerfile.
- Verify the result with a real image build and container test.

2. Common Dockerfile Instructions

Instruction	Purpose
FROM	Set base image
WORKDIR	Set working directory
COPY	Copy files into image
RUN	Run commands (install packages)
ENV	Set environment variables
USER	Set non-root user
EXPOSE	Document container port
CMD / ENTRYPOINT	Define the start command

3. Example: Initial Dockerfile (Needs Improvement)

Not Optimized

```
FROM python:3.11
WORKDIR /app
COPY . /app
RUN apt-get update && apt-get install -y \
    build-essential \
    curl \
    && pip install -r requirements.txt
EXPOSE 8000
CMD ["python", "app.py"]
```

Problems with this Dockerfile

- Uses a large base image
- Installs unnecessary packages (build-essential, curl)
- Runs as root (security risk)
- Does not use a .dockerignore
- All dependencies in one layer
- Image size is larger than needed

4. Improved Dockerfile (Optimized)

Better Practice

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN useradd -m appuser
USER appuser
EXPOSE 8000
CMD ["python", "app.py"]
```

Why This is Better

- Uses a smaller base image (slim)
- Installs only required Python packages
- Uses `--no-cache-dir` to reduce image size
- Creates and runs as a non-root user
- Copies files in a efficient order (better caching)
- Reduces final image size and attack surface

.dockerignore (Important)

```
.git
__pycache__/
*.pyc
.venv/
.env
node_modules/
tests/
*.log
```

This prevents unnecessary files from being copied into the image, keeping it smaller and more secure.

5. Build and Test the Image

1. Build the image
   `docker build -t myapp:latest .`
2. Check the image size
   `docker images | grep myapp`
3. Run the container
   `docker run -d -p 8000:8000 --name myapp myapp:latest`
4. Test the application
   `curl http://localhost:8000`

6. Compare Image Size

```
$ docker images | grep myapp
REPOSITORY    TAG       IMAGE ID       SIZE
myapp         old       a1b2c3d4       1.2GB
myapp         latest    e5f6g7h8       180MB
```

The optimized Dockerfile reduces the image size from 1.2GB to 180MB (85% smaller).

7. Ask AI to Review Your Dockerfile

- Paste your Dockerfile and ask for a review.
- Ask for specific suggestions to reduce image size.
- Request security best practices (non-root, minimal packages).
- Ask if any dependencies are unnecessary.
- Request a multi-stage build if applicable.

8. Junior Engineer Insight

AI can help you write a better Dockerfile, but you must understand the changes and verify the result. Always build and test the image yourself before using it.

9. Important Reminders

- Do not run containers as root.
- Avoid installing unnecessary packages.
- Use a .dockerignore file.
- Verify the image by building and testing it.
- Do not blindly accept AI suggestions.
- Consider using multi-stage builds for even smaller images.

9. Troubleshoot Failing Docker Containers

VERIQTA

1. What You Will Learn

- Investigate containers that exit immediately or cannot serve requests.
- Use docker ps -a, docker logs, and docker inspect.
- Diagnose a container with an incorrect startup command.
- Distinguish application failures from container configuration problems.

2. Common Docker Troubleshooting Commands

Command	Purpose
`docker ps -a`	List all containers (including exited ones)
`docker logs <container>`	View container logs
`docker logs -f <container>`	Follow logs in real time
`docker inspect <container>`	Get detailed container information
`docker exec -it <container> sh`	Access a running container shell
`docker start <container>`	Start an exited container
`docker rm <container>`	Remove a container

3. Check Container Status

```
$ docker ps -a
CONTAINER ID   IMAGE   COMMAND                  STATUS        PORTS     NAMES
a1b2c3d4e5f6   myapp   "python app.py"          Exited (1)    0.0.0.0   myapp
f6e5d4c3b2a1   nginx   "/docker-entrypoint..."  Up 2 hours    80/tcp    web
```

How to interpret STATUS

- **Up** – container is running
- **Exited (0)** – finished successfully
- **Exited (1)** – exited with an error
- **Restarting** – container keeps restarting
- **Created** – not started yet

4. View and Analyze Logs

```
$ docker logs myapp
Traceback (most recent call last):
  File "/app/app.py", line 5, in <module>
    app.run()
ModuleNotFoundError: No module named 'flask'
```

What the logs tell you

- Shows application errors, missing dependencies, and runtime issues.
- Use `docker logs -f <container>` to follow logs.
- Look for error messages, stack traces, and clues about what failed.

5. Inspect Container Details

```
$ docker inspect myapp | grep -A 5 "Config"
    "Config": {
        "Image": "myapp:latest",
        "Cmd": ["python","app.py"],
        "Env": [
            "ENV=production"
        }
```

What to check in docker inspect

- **Cmd** – is the startup command correct?
- **Env** – are required environment variables set?
- **WorkingDir** – is the application path correct?
- **Mounts** – are required files or volumes available?
- **RestartPolicy** – is the container restarting automatically?

6. Practical Example: Incorrect Startup Command

Failing Container (Wrong Command)

```
$ docker run --name myapp-bad \
    myapp "python ap.py"

$ docker logs myapp-bad
python: can't open file '/app/ap.py':
[Errno 2] No such file or directory
```

Correct Container (Fixed Command)

```
$ docker run --name myapp \
    myapp "python app.py"

$ docker logs myapp
 * Running on http://0.0.0.0:8000
 * Press CTRL+C to quit
```

Why it failed

- The command had a typo (ap.py instead of app.py).
- Container started, but the application crashed immediately.
- Logs show the exact error.

How to fix

- Correct the startup command.
- Rebuild or rerun the container.
- Verify by checking logs and container status.

7. Application vs Container Configuration

Issue Type	Examples	How to Identify	Next Step
Application failure	Missing package, code error, wrong config	Error messages in logs (stack trace, app error)	Fix code, install dependencies, update config
Container configuration problem	Wrong CMD, missing env, wrong volume, port not exposed	No application logs or immediate exit	Check Dockerfile, run command, environment variables, volumes

8. Junior Engineer Insight

Always check both application logs and container configuration. A container can fail because of an application error (e.g. missing package) or a configuration issue (e.g. wrong command or missing environment variable).

9. Important Reminders

- Always check logs before making changes.
- Verify the startup command and environment variables.
- Do not assume the container image is correct.
- Test the fix by running the container and checking logs, status, and application behavior.

10. Generate Kubernetes YAML and Validate It

VERIQTA

1. What You Will Learn

- Use AI to draft Deployments, Services, and resource configurations.
- Understand selectors, labels, ports, replicas, and readiness probes.
- Create a Deployment and Service for a simple web application.
- Validate manifests using kubectl apply --dry-run=client and inspect their structure before deployment.

2. Key Kubernetes Concepts

Concept	Purpose
Deployment	Manages replica pods and updates
Service	Exposes pods inside or outside the cluster
Labels	Key-value pairs to organize resources
Selectors	Match labels to select pods
Replicas	Number of pod copies
Ports	Container port and service port
Readiness probe	Checks if the application is ready to serve traffic
Resources	CPU and memory requests and limits

3. Example: Deployment YAML (AI Generated)

Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
  labels:
    app: webapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
    spec:
      containers:
      - name: webapp
        image: nginx:1.25
        ports:
        - containerPort: 80
        readinessProbe:
          httpGet:
            path: /
            port: 80
          initialDelaySeconds: 10
          periodSeconds: 5
```

4. Example: Service YAML (AI Generated)

Service

```yaml
apiVersion: v1
kind: Service
metadata:
  name: webapp-service
  labels:
    app: webapp
spec:
  type: ClusterIP
  selector:
    app: webapp
  ports:
  - port: 80
    targetPort: 80
    protocol: TCP
```

5. Validate the Manifests (Dry Run)

1. `kubectl apply --dry-run=client -f deployment.yaml`
   Validates the Deployment YAML without creating resources.
2. `kubectl apply --dry-run=client -f service.yaml`
   Validates the Service YAML without creating resources.
3. `kubectl explain deployment.spec`
   View the structure and available fields.
4. `kubectl explain service.spec`
   View the structure and available fields.

6. Inspect the Generated Resources

`kubectl get deployments`	List deployments
`kubectl get pods -l app=webapp`	List pods with label
`kubectl get services`	List services
`kubectl describe deployment webapp`	View detailed info
`kubectl describe service webapp-service`	View service details
`kubectl get pods -o wide`	See node and IP details

7. Common AI Mistakes to Check

- Using incorrect apiVersion or kind.
- Missing or wrong labels and selectors.
- Incorrect container port or service port.
- No readiness probe.
- Unrealistic resource requests/limits.
- Exposing the service publicly when not needed.
- Invalid YAML indentation or structure.

8. Junior Engineer Insight

AI can quickly generate correct Kubernetes YAML, but you must understand the components and always validate the manifests, review the configuration, and test in a safe environment before deploying to production.

9. Important Reminders

- Always validate YAML with --dry-run=client first.
- Check labels, selectors, ports, and resource values.
- Use readiness probes for reliable deployments.
- Start with a simple configuration and test.
- Review AI-generated YAML and understand every field.
- Do not apply unverified manifests to production.

11. Investigate Kubernetes Failures with AI

VERIQTA

1. What You Will Learn

- Diagnose CrashLoopBackOff, ImagePullBackOff, and unhealthy Pods.
- Collect evidence with kubectl describe pod and kubectl logs.
- Troubleshoot a container failing because of a missing configuration value.
- Distinguish symptoms from root causes before changing resources.

2. Common Pod Failure Statuses

Status	Meaning	Common Causes
CrashLoopBackOff	Container keeps crashing and restarting	Application error, missing config, bad command
ImagePullBackOff	Cannot pull the image	Wrong image name, no access, private registry issue
ErrImagePull	Failed to pull the image	Image not found or auth error
Pending	Pod not scheduled	Insufficient resources, node issues, taints/tolerations
Running (Unhealthy)	Pod is running but not ready	Readiness probe failing, app not responding

3. Check Pod Status

```
$ kubectl get pods
NAME                     READY   STATUS             RESTARTS   AGE
webapp-7d8c6f9d4-abc12   0/1     CrashLoopBackOff   5          2m
api-6f9c7b8d5-xyz34      1/1     Running            0          10m
worker-5c7db6d9-ptx9     0/1     ImagePullBackOff   0          3m
```

4. Describe the Pod for Details

```
$ kubectl describe pod webapp-7d8c6f9d4-abc12
Name:         webapp-7d8c6f9d4-abc12
Namespace:    default
Status:       Running
Containers:
  webapp:
    State:       Waiting
      Reason:    CrashLoopBackOff
    Last State:  Terminated
      Reason:    Error
      Exit Code: 1
      Started:   Mon, 26 Oct 2026 10:12:32
      Finished:  Mon, 26 Oct 2026 10:12:34
    Ready:       False
    Restart Count: 5
Events:
  Type     Reason   Age      From     Message
  Warning  BackOff  1m (x5)  kubelet  Back-off restarting failed container
```

5. Check Container Logs

```
$ kubectl logs webapp-7d8c6f9d4-abc12
Traceback (most recent call last):
  File "/app/app.py", line 10, in <module>
    config_value = os.environ["APP_CONFIG"]
KeyError: 'APP_CONFIG'
```

6. Identify the Root Cause

- The logs show a missing environment variable (APP_CONFIG).
- This causes the application to crash.
- The container keeps restarting, leading to CrashLoopBackOff.
- This is an application configuration issue, not a Kubernetes cluster problem.

7. Fix the Issue (Add Missing Configuration)

```yaml
# deployment.yaml (relevant section)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
spec:
  template:
    spec:
      containers:
      - name: webapp
        image: myapp:latest
        env:
        - name: APP_CONFIG
          value: "production"   # add the missing value
```

8. Apply and Verify

1. Apply the updated manifest
   `kubectl apply -f deployment.yaml`
2. Check the pod status
   `kubectl get pods`
3. View logs to confirm it is working
   `kubectl logs webapp-7d8c6f9d4-abc12`
4. Check readiness
   `kubectl describe pod webapp-7d8c6f9d4-abc12`

Pod should be Running and Ready with no more crashes.

9. Junior Engineer Insight

AI can help you analyze logs and error messages quickly, but you must understand the Kubernetes concepts to identify the real root cause. Always verify solutions, check the configuration, and confirm the pod is healthy after making changes.

10. Common Mistakes

- Only checking pod status without looking at logs.
- Changes without understanding the root cause.
- Assuming it is a cluster issue when it is an application issue.
- Ignoring events in kubectl describe pod.
- Not verifying that the fix actually solved the problem.

11. Important Reminders

- Always check both kubectl describe pod and kubectl logs.
- Read the events section for useful clues.
- Verify environment variables, config maps, and secrets.
- Understand the difference between symptoms and root causes.
- Test the application after making changes.

12. Review Terraform Infrastructure with AI

VERIQTA

1. What You Will Learn

- Use AI to explain Terraform resources, variables, dependencies, and plans.
- Understand how infrastructure changes can affect existing cloud resources.
- Review a Terraform plan that proposes replacing an AWS resource.
- Never approve infrastructure changes solely because AI describes them as safe.

2. Key Terraform Concepts

Concept	Purpose
Resource	Defines an infrastructure object (e.g. aws_instance, aws_s3_bucket)
Variable	Makes configuration reusable and flexible
Dependency	Controls the order of resource creation
Data source	Reads existing infrastructure information
State file	Tracks real infrastructure resources
Plan	Shows what will be created, changed or destroyed
Apply	Executes the approved changes
Modules	Organizes and reuses configuration

3. Example: Terraform Resource (AI Generated)

EC2 Instance

```
resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = var.instance_type
  tags = {
    Name        = "web-server"
    Environment = var.environment
  }
}
```

4. Variables Example

Variable

```
variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}
```

5. Review a Terraform Plan (Example Output)

```
$ terraform plan
Terraform will perform the following actions:

  # aws_instance.web must be replaced
-/+ resource "aws_instance" "web" {
      ~ instance_type = "t3.micro" -> "t3.medium"  # forces replacement
      ~ tags          = {
            Environment = "production"
        }
      + user_data     = (known after apply)
    }

Plan: 1 to add, 0 to change, 1 to destroy.
```

6. Understand What the Plan Means

- **+ Create** — A new resource will be created.
- **~ Update** — An existing resource will be modified in place.
- **− Destroy** — A resource will be destroyed.
- **−/+ Replace** — The existing resource will be destroyed and a new one will be created.

7. Ask AI to Explain the Plan

Example Prompt:

"Explain this Terraform plan output. What will change, what will be destroyed, and what are the potential risks for production?"

AI Explanation (Example)

- The instance_type change from t3.micro to t3.medium forces a replacement, so the existing EC2 instance will be destroyed and a new one will be created.
- This may cause downtime if the instance hosts a live application.
- Associated resources (e.g. EBS volumes, IP addresses) may also change or be lost depending on configuration.
- Review if downtime is acceptable and consider using a launch template or ASG for zero-downtime replacement.

8. Production Checks Before Approving

- Will this cause downtime or service interruption?
- Are dependent resources (load balancers, databases) affected?
- Is data at risk of being lost?
- Are there safer alternatives (e.g. in-place update, rolling update)?
- Have you tested the changes in a non-production environment?
- Do the changes align with your change management process?

9. Junior Engineer Insight

AI can help you understand Terraform plans quickly, but you must verify the impact yourself. Always review the full plan, understand the dependencies, and consider the production environment before approving any infrastructure changes.

10. Common AI Mistakes

- Assuming a plan is safe without reviewing the details.
- Missing that a resource will be replaced or destroyed.
- Ignoring the impact on dependent resources.
- Not considering downtime or data loss.
- Trusting AI's explanation without validating in your cloud console.

11. Important Reminders

- Always review the full terraform plan output.
- Understand what will be created, changed or destroyed.
- Check the impact on existing resources and services.
- Test changes in a staging environment first.
- Never approve infrastructure changes solely because AI describes them as safe.

13. Build a CI/CD Pipeline with AI Assistance

VERIQTA

1. What You Will Learn

- Use AI to draft a GitHub Actions workflow for a Python application.
- Connect checkout, dependency installation, tests, and build verification.
- Run automated tests whenever a pull request is opened.
- Validate workflow syntax, permissions, dependencies, and failure handling.

2. Key GitHub Actions Concepts

Concept	Purpose
Workflow	Automates your CI/CD process
Event	Triggers the workflow (e.g. pull_request)
Job	A set of steps that run on a runner
Step	An individual task (e.g. run tests)
Action	Reusable component (e.g. actions/checkout)
Runner	Virtual machine that executes the workflow
Permissions	Controls access to repository and resources
Secrets	Stores sensitive information (e.g. API keys)
Artifacts	Stores build outputs (e.g. test reports, packages)

3. Example: GitHub Actions Workflow (AI Generated)

.github/workflows/ci.yml

```yaml
name: Python CI
on:
  pull_request:
    branches: [ main ]
  push:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - naes: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
      - name: Run tests
        run: |
          pytest --maxfail=1 --disable-warnings
      - name: Verify build
        run: |
          python -m build
```

4. Explain the Workflow (AI Assisted)

1. **name:** Sets a descriptive name for the workflow.
2. **on:** Triggers the workflow on pull request and push to main branch.
3. **runs-on:** Uses an Ubuntu runner environment.
4. **Checkout:** Downloads your repository code.
5. **Set up Python:** Installs the specified Python version.
6. **Install dependencies:** Installs required packages from `requirements.txt`.
7. **Run tests:** Executes pytest to run automated tests.
8. **Verify build:** Builds the application to confirm it can be packaged.

5. Real Example: Pull Request Trigger

Python CI
Pull request #42 opened by user · 1 minute ago

Checkout code	3s
Set up Python	6s
Install dependencies	12s
Run tests	18s
Verify build	9s

All jobs completed successfully

6. Validate the Workflow

1. Check YAML syntax (optional)
   `yamllint .github/workflows/ci.yml`
2. Test the workflow locally with act (optional)
   `act -n`
3. Commit and push the workflow file
   ```
   git add .github/workflows/ci.yml
   git commit -m "Add CI workflow"
   git push
   ```
4. Open a pull request and check the Actions tab
   Verify that the workflow runs and tests are executed.

7. Failure Handling and Best Practices

- Use appropriate permissions (least privilege).
- Fail fast when tests fail (e.g. --maxfail=1).
- Use caching to speed up dependencies (e.g. actions/cache).
- Validate the workflow before merging.
- Review logs when a step fails.
- Use versioned actions (e.g. @v4, @v5).
- Keep secrets in GitHub Secrets, not in code.
- Consider additional steps (linting, security scanning, build artifacts).

8. Common AI Mistakes

- Generating incorrect or outdated action versions.
- Missing required permissions or dependencies.
- Using insecure practices (e.g. exposing secrets).
- Not handling failures or edge cases.
- Assuming the workflow is correct without testing.

9. Important Reminders

- Always validate the workflow syntax and test it.
- Review and understand each step before merging.
- Keep dependencies up to date.
- Check the Actions logs for any failures.
- Do not approve changes solely because AI says they are safe.

VERIQTA

14. Debug Broken CI/CD Pipelines

1. What You Will Learn

- Investigate failed jobs, missing dependencies, incorrect paths, and permission errors.
- Use AI to analyze the failed step and error output.
- Fix a pipeline failing during dependency installation.
- Reproduce the failure and confirm the fix with another pipeline run.

2. Common CI/CD Pipeline Failures

Error Type	Example Error Message	Common Cause
Missing package	ModuleNotFoundError: No module named 'flask'	Dependencies not installed or wrong Python version
Incorrect path	File not found: requirements.txt	Wrong working directory or file path
Permission error	Permission denied	Insufficient permissions to access file or resource
Authentication error	403 Forbidden	Invalid or missing token or credentials
Build failure	Command exited with code 1	Error in build script or failing tests

3. Example: Failed Pipeline (Dependency Installation)

FAILED

```
Run pip install -r requirements.txt
Collecting flask
ERROR: Could not find a version that satisfies
  the requirement flask==2.3.0 (from versions: none)
ERROR: No matching distribution found for flask==2.3.0
Error: Process completed with exit code 1.
```

4. Analyze the Error with AI

Example Prompt:

"This GitHub Actions step is failing. Here is the error output: [paste error]. What is the likely cause and how can I fix it?"

- AI can help identify the root cause.
- It can suggest the correct package version, Python version, or updated commands.
- Always review and test the suggested fix.

5. Fix the Pipeline (Updated Workflow)

Fixed

```yaml
name: Python CI
on:
  pull_request:
    branches: [ main ]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
      - name: Run tests
        run: pytest
```

6. Verify the Fix (Successful Run)

All jobs completed successfully

```
✓ Set up job                    3s
✓ Checkout code                 4s
✓ Set up Python                 6s
✓ Install dependencies          12s
  ▸ Run pip install -r requirements.txt
    Collecting flask==2.3.0
    Successfully installed flask-2.3.0 ...
✓ Run tests                     8s
  ================ 5 passed in 4.21s ================
✓ Complete job                  1s
```

7. Reproduce and Confirm

1. Re-run the pipeline to confirm the fix.
2. Test both pull request and main branch triggers.
3. Verify that dependencies, tests, and build steps complete.
4. Check logs to ensure there are no hidden warnings or errors.

8. Additional Debugging Commands

```
# View full logs for a job
gh run view <run-id> --log
# Rerun a failed workflow
gh run rerun <run-id>
# View workflow file locally
cat .github/workflows/ci.yml
# Check Python and pip versions
python --version && pip --version
```

Common Mistakes

- Using the wrong Python version.
- Missing or outdated dependencies.
- Wrong file paths (e.g. requirements.txt).
- Not caching dependencies (slower builds).
- Ignoring error messages in logs.
- Assuming AI's fix is correct without testing.

10. Important Reminders

- Always read the full error output.
- Provide relevant logs and workflow file when asking AI.
- Test the fix with another pipeline run.
- Use caching to speed up builds.
- Review permissions and secrets.
- Confirm tests and build steps pass before merging.

15. Analyze Production Logs with AI

VERIQTA

1. What You Will Learn

- Understand how AI helps group errors, summarize patterns, and suggest hypotheses.
- Use journalctl, grep, and structured application logs.
- Investigate repeated HTTP 500 errors after a deployment.
- Separate confirmed log evidence from AI-generated explanations.



2. Useful Commands for Log Analysis

Command	Purpose
journalctl -u app.service	View recent logs for a service
journalctl -u app.service -f	Follow logs in real time
grep -i "error" app.log	Search for errors (case insensitive)
grep "500" app.log	Find HTTP 500 errors
tail -n 100 app.log	View the last 100 lines
less app.log	Read logs interactively
journalctl --since "1 hour ago"	View logs from a specific time
grep -E "(error|exception)" app.log	Search multiple patterns

3. Example: Investigate HTTP 500 Errors

```
$ journalctl -u myapp.service --since "1 hour ago" | grep "500"
Oct 26 10:12:31 server myapp[1234]: ERROR 500 GET /api/users
Oct 26 10:12:32 server myapp[1234]: ERROR 500 POST /api/login
Oct 26 10:12:33 server myapp[1234]: ERROR 500 GET /api/users
Oct 26 10:12:40 server myapp[1234]: ERROR 500 POST /api/login
Oct 26 10:12:45 server myapp[1234]: ERROR 500 GET /api/users
```

4. Analyze the Pattern with AI



Example Prompt:

"These are application logs showing repeated HTTP 500 errors after a deployment. Please analyze the pattern, group similar errors, and suggest possible root causes."

Example AI Analysis (Summary)

- Most 500 errors occur on /api/users and /api/login.
- Errors started immediately after the deployment.
- The logs show a database connection error.
- Likely cause: missing or incorrect environment variable for the database URL.
- Suggested next steps: check configuration, verify secrets, and confirm database connectivity.

5. Check Detailed Error Logs

```
$ tail -n 20 /var/log/myapp/app.log
2026-10-26 10:12:31 [ERROR] Exception in request
Traceback (most recent call last):
  File "/app/main.py", line 42, in get_users
    users = db.query("SELECT * FROM users").all()
  File "/app/db.py", line 15, in query
    raise ConnectionError("DB connection failed")
ConnectionError: DB connection failed
2026-10-26 10:12:32 [ERROR] 500 Internal Server Error:
GET /api/users
```

6. Verify the Root Cause

```
$ echo $DB_URL
```

Environment variable is missing

```
$ env | grep DB
$ cat .env
  # DB_URL is not set
```

Configuration value is missing

7. Fix the Issue (Set the Missing Configuration)

```
# Update the deployment configuration
env:
  - name: DB_URL
    value: "postgres://user:password@db:5432/appdb"

# Redeploy the application
kubectl apply -f deployment.yaml
```

8. Confirm the Fix

```
$ journalctl -u myapp.service --since "10 minutes ago" | grep "500"
```

No 500 errors found (after the fix)

```
$ tail -n 10 /var/log/myapp/app.log
2026-10-26 10:25:11 [INFO] GET /api/users 200
2026-10-26 10:25:12 [INFO] POST /api/login 200
2026-10-26 10:25:15 [INFO] GET /api/users 200
```

9. Key Takeaways



- Use real logs to understand what actually happened.
- AI can help identify patterns and suggest hypotheses.
- Always verify AI explanations with log evidence.
- Fix the root cause, not just the symptom.
- Confirm the issue is resolved with another deployment and log check.

10. Common Mistakes



- Relying only on AI explanations without checking logs.
- Looking at a small portion of logs.
- Ignoring time range and context.
- Misinterpreting errors that are not the real cause.
- Changing configuration without confirming the the evidence.
- Assuming a fix works without verifying in production.

11. Important Reminders



- Always collect and review the actual logs.
- Clearly separate confirmed evidence from AI suggestions.
- Use appropriate time ranges.
- Check related services (database, cache, external APIs).
- Verify the fix with real traffic and monitoring.

16. Use AI with Metrics, Alerts, and Traces

VERIQTA

1. What You Will Learn

- Connect CPU, memory, request latency, error rate, and service dependencies.
- Understand what Prometheus, Grafana, and distributed traces reveal.
- Investigate rising API latency while CPU usage remains normal.
- Learn why a single metric rarely proves the root cause.

Prometheus

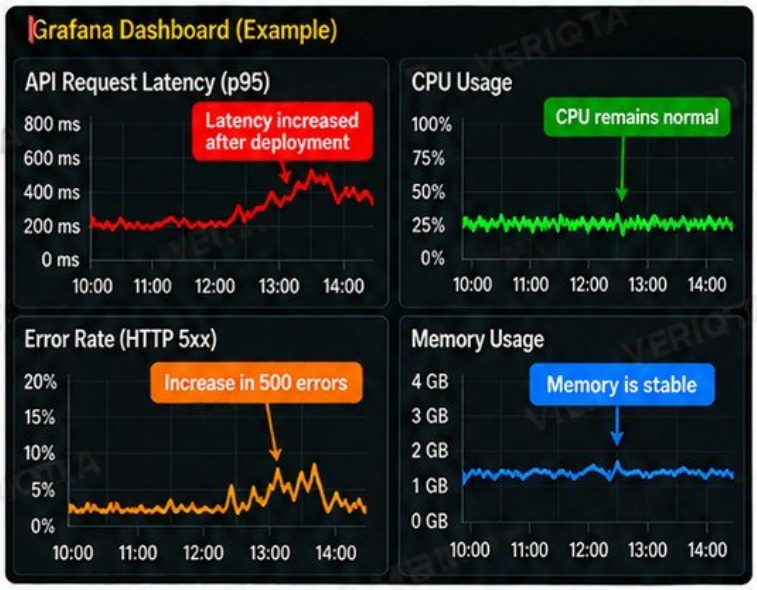
Grafana

Jaeger Traces

2. Key Tools and What They Show

Tool	What It Shows	Example Use
Prometheus	Collects and stores metrics (CPU, memory, request rate, latency, errors)	Monitor application and infrastructure metrics
Grafana	Visualizes metrics with dashboards and alerts	Identify trends, spikes, and anomalies
Jaeger	Distributed traces to see request flow across services	Find where latency happens in a microservices architecture
Alerting (Alerts)	Notifies when metrics cross defined thresholds	Get alerted before users are impacted

3. Example: Investigate Rising API Latency



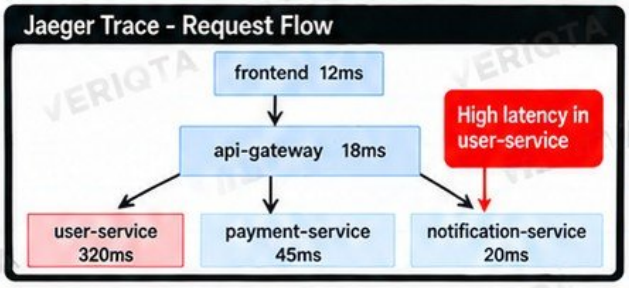
4. Check Metrics with Prometheus (Example)

```
# API request latency (p95)
histogram_quantile(0.95,
  rate(http_request_duration_seconds_bucket[5m]))

# Error rate (HTTP 5xx)
  rate(http_requests_total{status=~"5.."}[5m])

# CPU usage
  rate(container_cpu_usage_seconds_total[5m])
```

5. Investigate Traces with Jaeger (Example)



6. Use AI to Analyze the Data

Example Prompt:

"Here are my Prometheus and Grafana metrics. API latency increased but CPU and memory look normal. Here are the logs and traces. What are the possible causes and how can I investigate further?"

Example AI Analysis (Summary)

- Traces show user-service is slow (320ms).
- Latency caused by a slow database query.
- CPU is normal because the service is waiting on the database.
- Check database metrics, connection pool, and query performance.
- Consider adding caching or optimizing the query.

7. Common Root Causes of Latency (Even with Normal CPU)

1. Slow database queries or external API calls
2. High network latency between services
3. Increased queueing or thread pool exhaustion
4. Garbage collection pauses
5. Lock contention or inefficient code
6. Dependency service degradation

8. Validate and Confirm the Fix

- Identify the root cause from metrics and traces
- Apply the fix (e.g. optimize database query)
- Monitor metrics to confirm latency is reduced
- Check that error rate returns to normal
- Ensure alerts are no longer firing

9. Important Lessons

- A single metric rarely proves the root cause.
- Always look at metrics, logs, and traces together.
- AI can help you find patterns and suggest hypotheses, but you must confirm them.
- Understand your service dependencies.
- Use alerts to detect issues early.

10. Common Mistakes

- Assuming high latency is always caused by high CPU.
- Ignoring traces and focusing only on metrics.
- Not checking dependency services.
- Relying only on AI explanations without verifying the evidence.
- Not setting proper alerts.

17. Use AI for DevSecOps Without Exposing Secrets

VERIQTA

1. What You Will Learn

- Review dependencies, Dockerfiles, CI/CD workflows, and infrastructure configurations.
- Use scanner findings as evidence for AI-assisted explanations.
- Investigate a container image vulnerability and evaluate remediation.
- Understand how to use AI for security analysis without exposing secrets.
- Remember that AI suggestions do not replace security scanning, access controls, or policy enforcement.

2. Key DevSecOps Areas to Review

Area	What to Check
Dependencies	Outdated or vulnerable packages (e.g. pip, npm, Maven)
Dockerfile	Base image, unnecessary packages, root user, exposed ports
CI/CD Workflow	Secrets handling, permissions, hardcoded credentials, untrusted actions
Infrastructure (IaC)	Open security groups, overly permissive IAM policies, public resources
Container Images	Known vulnerabilities (CVEs) and misconfigurations
Configuration Files	Sensitive data, exposed endpoints, insecure defaults

3. Example: Scan a Container Image (Trivy)

```
$ trivy image python:3.11
```

Vulnerabilities found

VULNERABILITY	SEVERITY	PACKAGE	VERSION	FIXED IN
CVE-2024-1234	HIGH	openssl	3.0.8	3.0.13
CVE-2024-5678	MEDIUM	libxml2	2.9.14	2.9.14+dfsg-1.5
CVE-2023-9999	LOW	zlib	1.2.11	1.2.13

Total: 3 (HIGH: 1, MEDIUM: 1, LOW: 1)

4. Ask AI to Explain the Findings

Example Prompt:

"Here are the Trivy scan results for the python:3.11 image. Please explain the risks of each vulnerability and recommend how to fix them. Do not invent additional vulnerabilities, only use the provided results."

Example AI Response (Summary)

- CVE-2024-1234 (openssl): High risk. Can allow remote code execution. Upgrade to 3.0.13.
- CVE-2024-5678 (libxml2): Medium risk. Can cause XML parsing attacks. Upgrade to 2.9.14+dfsg-1.5.
- CVE-2023-9999 (zlib): Low risk. Potential denial of service. Upgrade to 1.2.13.

5. Fix the Dockerfile (Use a Patched Base Image)

```
1  FROM python:3.11-slim
2  WORKDIR /app
3  COPY requirements.txt .
4  RUN pip install --no-ache-dir -r requirements.txt
5  COPY . .
6  RUN adduser --disabled-password --gecos "" appuser
7  USER appuser
8  EXPOSE 8000
9  CMD ["python", "app.py"]
```

Updated and more secure

6. Re-scan the Image to Verify the Fix

```
$ trivy image myapp:latest
No vulnerabilities found
```

Image is clean after remediation

7. Review CI/CD Workflow for Secrets Issues

```
1  name: Python CI
2  on: [pull_request]
3  jobs:
4    test:
5      runs-on: ubuntu-latest
6      steps:
7        - uses: actions/checkout@v4
8        - name: set up Python
9          uses: actions/setup-python@v5
10       - name: Install dependencies
11         run: pip install -r requirements.txt
```

Do not hardcode secrets in workflows

8. Security Best Practices When Using AI

- Do not share real secrets, API keys, passwords, or tokens.
- Redact or replace sensitive values with placeholders.
- Use scanner results, logs, or configuration snippets as input.
- Verify AI suggestions with official documentation.
- Always enforce security policies, access controls, and code reviews.
- Use tools like Trivy, Snyk, Dependency-Check, Checkov, or GitHub Advanced Security.
- Remember that AI can explain findings, but it does not perform the actual security scan or enforce your policies.

10. Common Mistakes

- Sharing real secrets or credentials with AI.
- Ignoring scanner findings.
- Using outdated base images.
- Relying only on AI explanations without verification.
- Running containers as root.

10. Important Reminders

- AI suggestions do not replace security scanning, access controls, or policy enforcement.
- Always validate fixes by re-running scans and testing.
- Keep dependencies, base images, and configurations up to date.
- Apply the principle of least privilege.
- Treat security as a continuous process, not a one-time check.

18. Investigate a Production Incident with AI

VERIQTA

1. What You Will Learn

- Work through a realistic incident where an API starts returning errors after deployment.
- Correlate deployment changes, application logs, metrics, and service health.
- Ask AI to produce competing hypotheses and safe diagnostic steps.
- Identify what evidence would justify a rollback and how to verify recovery.

2. Incident Scenario Overview

Service	User API (microservice)
Change	New version deployed at 14:05 UTC
Symptom	API starts returning 5xx errors
Impact	Users cannot log in or access key features
Goal	Use logs, metrics, deployment info, and AI assistance to find the root cause and recover safely

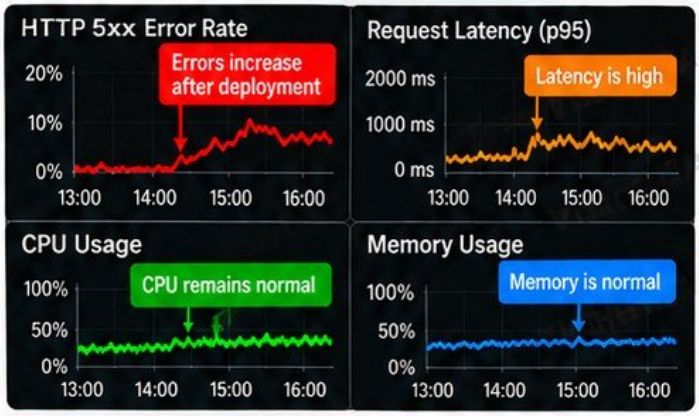
3. Check Recent Deployments

```
# View recent deployments (Kubernetes example)
kubectl rollout history deployment user-api
```

REVISION	CHANGE-CAUSE	DATE
8	Update to v1.3.0	2026-10-26 14:05:32
7	Update to v1.2.0	2026-10-24 10:12:11
6	Update to v1.1.0	2026-10-20 09:03:55

New version deployed at 14:05

4. Check Service Health and Metrics



5. Analyze Application Logs

```
# Check recent logs for errors
kubectl logs -n prod deployment/user-api --since=1h
2026-10-26 14:06:12 ERROR Database connection failed
2026-10-26 14:06:13 ERROR Timeout while calling auth-service
2026-10-26 14:06:14 ERROR 500 Internal Server Error
2026-10-26 14:06:15 ERROR Connection pool exhausted
2026-10-26 14:06:16 ERROR 500 Internal Server Error
```

Repeated errors after deployment

6. Ask AI for Competing Hypotheses

Example Prompt:

"Here are the logs, metrics, and deployment details for my API. The service started returning 500 errors after deployment. Please give 3–5 possible root causes and safe diagnostic steps to investigate."

Example AI Response (Hypotheses)

1. Database connection issue (e.g. new configuration or secret)
2. Incompatible dependency in the new version
3. Increased latency from downstream service (auth-service)
4. Resource limits causing connection pool exhaustion
5. Code change introducing a bug in the authentication flow

7. Investigate a Hypothesis (Example: DB Connection)

```
# Check database connectivity from the pod
kubectl exec -it user-api-xxxxx -- nc -zv db.prod.svc 5432
Connection to db.prod.svc 5432 port [tcp/postgresql] failed:
Connection timed out

# Check environment variables
kubectl exec -it user-api-xxxxx -- env | grep DB_
DB_HOST=db.prod.svc
DB_PORT=5432
```

Database connection failing after deployment

8. Determine and Execute the Fix

1. Confirm the root cause with evidence (logs, metrics, tests).
2. Apply the fix (e.g. correct DB configuration or rollback).
3. Monitor metrics and logs after the fix.
4. Verify the service is healthy and errors return to normal.

9. What Justifies a Rollback?

- Clear correlation between the new deployment and the incident.
- Consistent errors (e.g. 5xx) across multiple instances.
- Confirmed issue in the new code, configuration, or dependency.
- No quick safe fix available.
- Rollback is likely to restore service based on previous stable version.

10. Common Mistakes

- Relying only on AI without verifying actual evidence.
- Ignoring metrics and focusing only on logs.
- Changing multiple things at once.
- Assuming the first hypothesis is correct.
- Skipping verification after the fix.

11. Important Reminders

- Always use real data (logs, metrics, deployment info) when asking AI.
- Validate AI suggestions before making changes.
- Perform safe diagnostic steps first.
- Document findings and actions.
- Confirm full recovery (metrics, logs, user experience) after the fix.

19. Build a Guardrailed AI DevOps Assistant

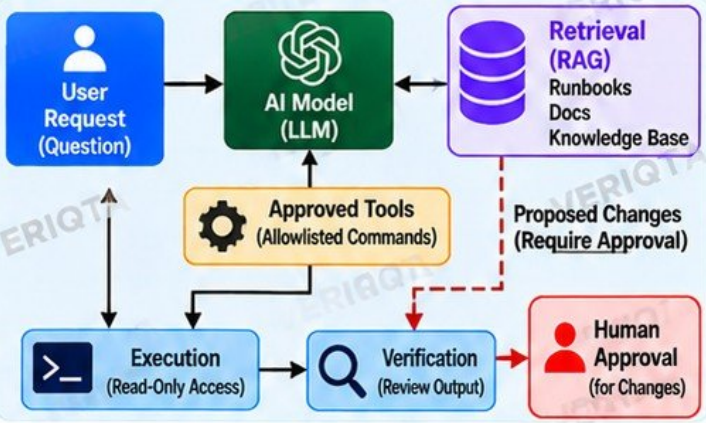
VERIQTA

1. What You Will Learn

- Understand the architecture: user request, AI model, approved tools, execution, and verification.
- Explore retrieval-augmented generation (RAG) for internal runbooks and documentation.
- Design read-only access, allowlisted commands, audit logs, and human approval for changes.
- Build an assistant that retrieves Kubernetes diagnostic information without modifying the cluster.



2. Architecture Overview



3. Retrieval-Augmented Generation (RAG)



- Use internal runbooks, documentation, and past incidents as context for AI responses.
- AI searches relevant content instead of relying only on general knowledge.
- Helps produce accurate, environment-specific answers.

4. Allowlisted Read-Only Kubernetes Commands

```
# Example: Safe read-only commands
kubectl get pods -A
kubectl get svc -A
kubectl get deployments -A
kubectl describe pod <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace> --tail=100
kubectl top pods -A
kubectl get events -A --sort-by=.lastTimestamp
```

These commands do not modify the cluster.

5. Example: AI Assistant Workflow (Read-Only)



User Request:
"Show me information about the pod user-api-7d9f in the prod namespace. Include pod status, recent logs, and any related events."



AI Assistant (using allowlisted tools):

1. Run kubectl get pod user-api-7d9f -n prod
2. Run kubectl describe pod user-api-7d9f -n prod
3. Run kubectl logs user-api-7d9f -n prod --tail=100
4. Run kubectl get events -n prod --field-selector involvedObject.name=user-api-7d9f

6. Example Output from the Assistant

```
# Pod status
$ kubectl get pod user-api-7d9f -n prod
NAME           READY   STATUS    RESTARTS   AGE
user-api-7d9f  1/1     Running   0          2h

# Recent logs (last 10 lines)
$ kubectl logs user-api-7d9f -n prod --tail=10
2026-10-26 14:12:01   INFO    Starting application
2026-10-26 14:12:03   INFO    Connected to database
2026-10-26 14:12:05   INFO    API server listening on port 8080
2026-10-26 14:15:12   ERROR   Timeout while calling auth-service
2026-10-26 14:15:12   ERROR   Retrying request (1/3)
2026-10-26 14:15:15   ERROR   Request failed: 504 Gateway Timeout
```

Assistant retrieves information without making changes.

7. Security and Guardrails

1. Use read-only access (no write permissions).
2. Allowlist only safe, necessary commands.
3. Restrict access to specific namespaces and resources.
4. Log all AI requests, commands, and outputs (audit logs).
5. Require human approval for any changes.
6. Prevent the AI from exposing secrets, credentials, or sensitive data.
7. Use RAG with internal runbooks and policies.
8. Monitor and review usage regularly.

8. Key Takeaways

- AI can be a powerful DevOps assistant when properly guarded.
- Use RAG for accurate, environment-specific answers.
- Read-only access and allowlisted commands reduce risk.
- Always verify AI output before taking action.
- Human approval is required for any changes to production.

9. Common Mistakes



- Giving the AI full cluster access.
- Allowing unrestricted shell commands.
- Sharing secrets, tokens, or sensitive data with the AI.
- Relying on AI output without verification.
- Skipping audit logs.
- Using AI to make production changes without approval.

10. Important Reminders

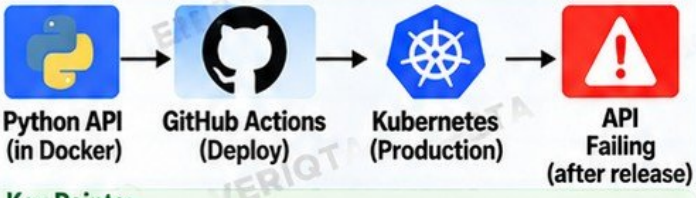


- AI suggestions do not replace security policies or access controls.
- Always use read-only access unless a change is explicitly approved.
- Verify information using official tools and documentation.
- Keep audit logs for accountability.
- Treat the AI as an assistant, not an autonomous operator.

20. Final Practical Lab: AI-Assisted Production Recovery

VERIQTA

1. Scenario Overview



Key Points:

- A Python API runs in Docker.
- Changes are deployed through GitHub Actions to Kubernetes.
- After a new release, the API starts returning errors.
- We will investigate, use AI to analyze findings, and recover safely.

2. Collect Deployment Status and Evidence

```
# Check deployment status
kubectl get deployments -n prod
kubectl get pods -n prod
kubectl get events -n prod --sort-by=.lastTimestamp

# Check recent rollout history
kubectl rollout history deployment/api -n prod

# Check service and endpoints
kubectl get svc -n prod
kubectl get endpoints -n prod
```

3. Check Container Logs and Application Health

```
# View pod logs
kubectl logs -n prod deployment/api
kubectl logs -n prod -l app=api
--tail=200

2026-10-26 12:31:18 ERROR Database
connection failed
2026-10-26 12:31:18 ERROR could not
connect to db:5432
2026-10-26 12:31:19 ERROR Retrying
in 5 seconds...
2026-10-26 12:31:24 ERROR Max
retries exceeded
```

```
# Check application health
kubectl port-forward svc/api 8080:80
curl -v http://localhost:8080/health

> GET /health HTTP/1.1
> Host: localhost:8080
< HTTP/1.1 500 Internal Server Error
< content-type: application/json
{
  "status": "error",
  "message": "Database connection
failed"
}
```

4. Use AI to Interpret Findings

Example Prompt:

"Here are the deployment details, logs, and metrics for my Python API running on Kubernetes. The API returns 500 errors after the new release. Please analyze the evidence, suggest possible root causes, and recommend safe diagnostic steps."

Example AI Analysis (Summary)

1. Most likely cause: database connection failure due to incorrect configuration in the new release.
2. Other possible causes: wrong secret, network policy, service not reachable, or database overloaded.
3. Recommended next steps: verify environment variables, check database service health, compare with previous version.

5. Compare Hypotheses

Hypothesis	Evidence For	Evidence Against	Next Step
Incorrect DB configuration	Logs show connection failed to db:5432	DB pod is running	Check env vars and secrets
Database service down	Connection timeout, 500 errors	DB pod healthy and reachable	Test connectivity from API pod
Network policy blocking	Timeout to DB	Policy rules allow traffic	Verify network policies
New code bug	Errors started after new release	No code errors in logs	Compare with previous version

6. Validate the Fix in a Controlled Environment

```
# Test the fix in a staging environment
kubectl set env deployment/api -n staging \
DB_HOST=db.staging.svc.cluster.local

kubectl rollout restart deployment/api -n staging
kubectl get pods -n staging
curl -v http://staging-api.example.com/health
```

Verify API returns 200 OK in staging first

7. Redeploy or Roll Back

Option 1: Roll Back to Previous Version

```
kubectl rollout undo deployment/api -n prod
kubectl get pods -n prod
kubectl rollout status deployment/api -n prod
```

Option 2: Redeploywith the Fix

```
kubectl set env deployment/api -n prod \
DB_HOST=db.prod.svc.cluster.local
kubectl rollout restart deployment/api -n prod
kubectl rollout status deployment/api -n prod
```

8. Verify Service Health After Recovery

- Check pod status: kubectl get pods -n prod
- Check API health endpoint (should return 200 OK)
- Monitor logs for errors
- Confirm metrics (error rate and latency return to normal)
- Verify user traffic and core functionality

9. Produce a Short Incident Report

- Timeline: key events (deployment, errors, investigation, recovery)
- Evidence: logs, metrics, kubectl output, and screenshots
- Root cause: clear explanation of what went wrong
- Recovery: actions taken and results
- Preventive actions: how to avoid similar issues in the future

10. Common Mistakes

- Skipping evidence collection
- Relying only on AI explanations
- Making changes in production without testing
- Ignoring metrics and service dependencies
- Not verifying that the service is fully recovered

11. Important Reminders

- Always verify evidence before action.
- Test the fix in a controlled environment.
- Have a rollback plan ready.
- Monitor the service after recovery.
- Document the incident clearly.